ENLANG FOR DEVELOPERS

The Definitive Guide to Building Full-Stack Software with Natural English

Author: Spandan Prayas Patra

Version: 2.0.0 — Enterprise Specification Edition

Publisher: Universal EnLang Foundation Press

> *"Programming should be as clear, expressive, and frictionless as human thought. EnLang makes natural English a first-class, production-grade programming language across the entire software stack."*
>
> — ***Spandan Prayas Patra***

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION & PHILOSOPHICAL ARCHITECTURE

EnLang is a deterministic, multi-target compiler and runtime engine created by Spandan Prayas Patra to convert natural English into 1:1 clean, production-grade native targets: Python 3, HTML5, CSS3, JavaScript ES6+, and SQL.

CHAPTER 5: FUNCTIONS, ACTIONS, INTERFACES & ASYNC / AWAIT

EnLang provides flexible, natural syntaxes for function declarations, invocations, and return values, while remaining 100% backward compatible with standard function syntax.

5.1 Function & Action Declarations

Style A: Modern Developer Shorthand (`fn`)

```
fn add_numbers(a, b):
    return a plus b

set total to add_numbers(10, 20)
display total
```

Style B: Signature EnLang Natural Expression (`define function` / `action`)

```
define function print_rec taking n:
    if n is greater than 10 then:
        return
    display n
    run print_rec with (n plus 1)

run print_rec with 1
```

Style C: Action Phrase (`action <name> taking <args>:`)

```
action calculate_tax taking price, rate:
    result is price times rate

call calculate_tax with (1000, 0.18) and store in tax
display tax
```

Style D: Legacy Standard (`function <name>(<args>):`)

```
function greet(name):
    display "Hello, " plus name

greet("Spandan")
```

5.2 Invocation / Call Keywords Summary

Functions can be invoked naturally using any of the following verbs:

- `run <name> with <args>` (e.g. `run print_rec with 1`)
- `call <name> with <args> and store in <var>` (e.g. `call calculate_tax with (1000, 0.18) and store in total`)
- `do <name> with <args>` (e.g. `do greet with "Spandan"`)
- `<name>(<args>)` (Direct call)

5.3 Return Value Keywords

- `return <expr>`
- `result is <expr>`
- `give back <expr>`

APPENDIX A: UNIVERSAL SYNTAX REFERENCE

Category	EnLang Syntax Example	Native Transpilation
Function (Fn)	`fn add(a, b): return a plus b`	`def add(a, b): return a + b`
Function (Natural)	`define function greet taking name:`	`def greet(name):`
Action	`action calc taking price, rate:`	`def calc(price, rate):`
Call & Store	`call calc with (100, 0.1) and store in t`	`t = calc(100, 0.1)`
Run Action	`run print_numbers with 1`	`print_numbers(1)`
Return Result	`result is a plus b` / `give back a plus b`	`return a + b`
